Python workflow for interactive projects (Colab + Github + Drive). Kaggle use case.

Leverage free cloud resources for your Jupyter Notebooks

Aleix López Pascual · Follow
Published in Analytics Vidhya · 8 min read · Oct 11, 2020

Jupyter Notebooks have become one of the most used tools for Python development in Data Science [1]. They are highly preferred by many data scientists due to their user-friendly interface and interactive computational environment.

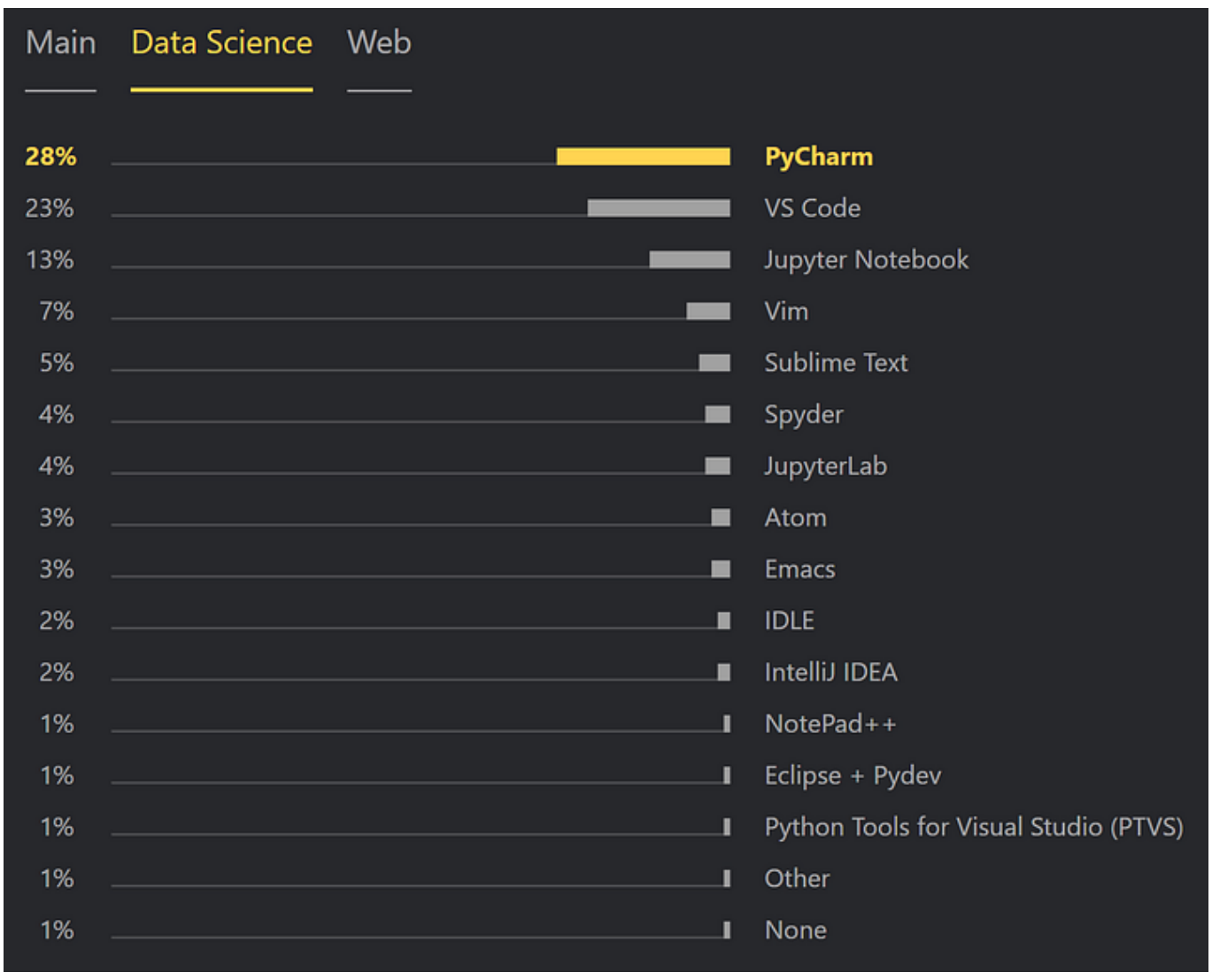


Image from JetBrains

Project Jupyter has been offering 100% open-source software in order to make Jupyter Notebooks accessible for everyone since 2014 [2]. However, these notebooks have limitations:

- **Jupyter Notebooks run on your local machine**, making the computational power available to you entirely dependent on your computer's CPU/GPU/RAM/etc. specs. While most laptops are more than

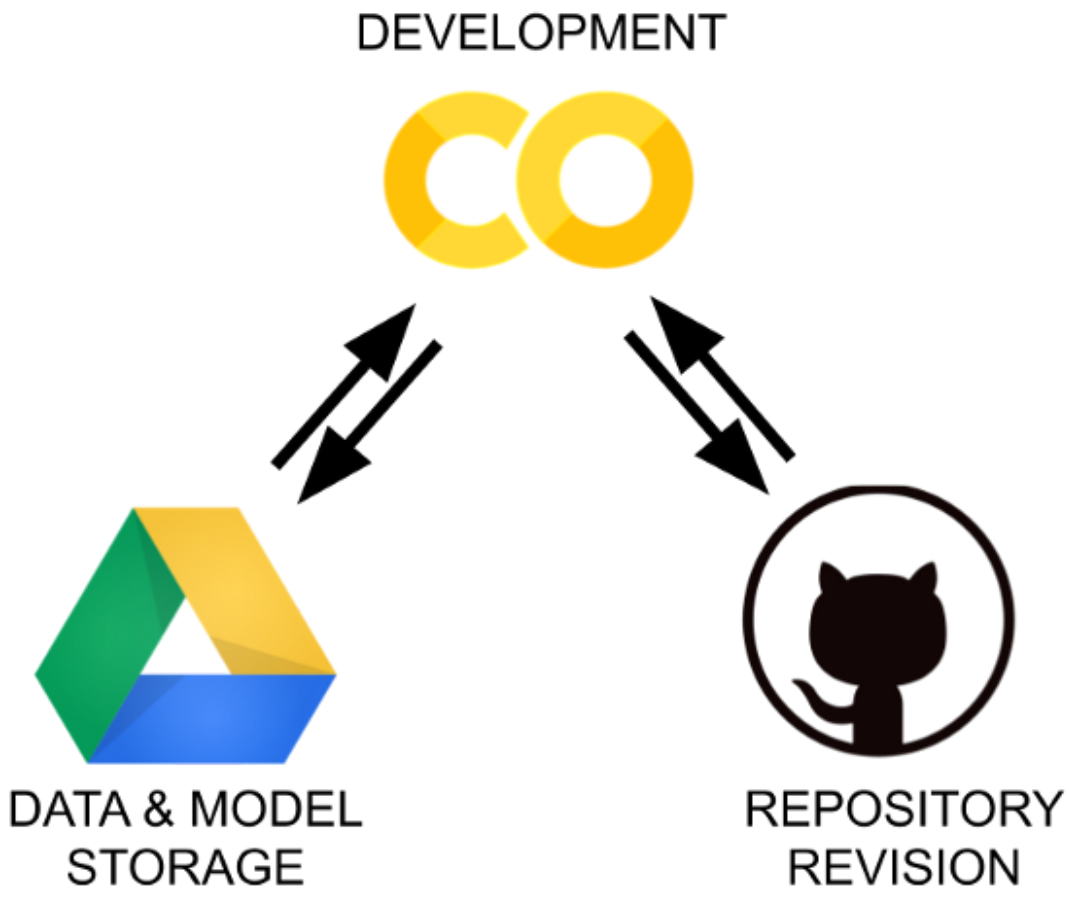
enough for the basic tasks one encounters when starting out in data science, you might quickly hit barriers once you start doing machine learning tasks (especially deep learning) on your local machine.

- **Jupyter Notebooks do not come with version control per se.** It is right that you can upgrade your Jupyter Notebooks to JupyterLab, and from there install the extensions @jupyterlab/github and @jupyterlab/git. I have used that before, but today I will show you a better option.

Google Drive + Google Colab + GitHub

In order to overcome the aforementioned limitations we will use the following combination:

- **Google Colab:** Colaboratory is a free Jupyter notebook environment that runs in the cloud and stores its notebooks on Google Drive. Ie, we will use Google Colab to develop our Jupyter Notebooks without depending on the computational power of our laptops. It can also be used as a shell to run bash and git commands.
- **Google Drive:** When we use Google Colab, the code is executed in a virtual machine private to our Google account. These virtual machines are deleted when idle for a while, and have a maximum lifetime of 12 hours. Therefore it is not ideal to store our work there since it will eventually be lost. The solution that we propose here is to store your output (weights, csvs…) in a cloud storage hosting. As we all know, Google Drive is the cloud storage provided by Google. It provides 15 GB of free storage and it is the default storage system when using Colab.
- **GitHub:** A code hosting platform for version control and collaboration. It's a good practice to use version control and branch strategy even when working with Jupyter Notebooks.



The three parts of our infrastructure

Other cloud-based notebook services

Google Colab is not the only free cloud service for Jupyter Notebooks. These are the most relevant to me:

- Google Colaboratory
- Kaggle Kernels
- Microsoft Azure Notebooks
- Datalore
- Binder
- CoCalc

All of them are completely free, they do not require you to install anything in your local machine, and they give you access to a Jupyter-like environment.

I personally like Colab because of the following:

- GPUs often include Nvidia K80s, T4s, P4s and P100s. Their availability varies over time. [3]
- TPU v2–8 [4]
- Idle cut-off 90 minutes
- Maximum 12 hours

On the other hand, we also have the Kaggle Kernels [5]:

- Nvidia Tesla P100 GPU

- TPU v3–8
- Idle cut-off 60 minutes
- Maximum 9 hours using GPUs
- Maximum 3 hours using TPUs

As I am an active participant of Kaggle competitions, I am always combining both. This can be managed easily using features toggles. More on that later.

Setting up the workflow

In this section, I am going to guide you through the different steps you must complete in order to configure the workflow. First things first, I am assuming that you already have a Goggle and Github accounts and you are familiar with them. Otherwise you will not be able to complete the process.

If you are ready, just create a Google Colaboratory Notebook using your Google account. You will have to copy the following snippets of code as cells step by step.

1. Feature toggles and path

```python
COLAB = True
KAGGLE = False
DOWNLOAD_DATA = True
SAVE_TO_GITHUB = False
GIT_REPOSITORY = "osic-pulmonary-fibrosis-progression"
FILE_NAME = "baseline.ipynb"

if COLAB:
    PARENT_DIRECTORY_PATH = "/content"
    # In case you want to clone in your drive:
    # PARENT_DIRECTORY_PATH = "/content/gdrive/My Drive"
    PROJECT_PATH = PARENT_DIRECTORY_PATH + "/" + GIT_REPOSITORY
    %cd "{PARENT_DIRECTORY_PATH}"
```

medium_python_workflow_for_interactive_projects_0.py hosted with ❤ by GitHub view raw

The above snippet defines the feature toggles that we will activate or deactivate depending in our use case. It has several of them since I have tried to make it as general as possible.

As you can observe, I will be using my private repository osic-pulmonary-fibrosis-progression as an example. You should put the name of your public or private repository of interest. Same for FILE_NAME.

Notice that there is the possibility to deactivate both COLAB and KAGGLE toggles. This should be done in case we want to run the notebook locally using our IDE or editor of preference (e.g. PyCharm). This is beneficial in certain situations. I will comment more on that at the end of the article.

2. Linking personal Google Drive storage with Google Colab

```
if COLAB:
    %cd /content
    from google.colab import drive
    drive.mount('/content/gdrive')
```

medium_python_workflow_for_interactive_projects_1.py hosted with ❤ by GitHub view raw

Here we mount Google Drive to Colab. Mounting is the process by which the OS makes files and directories of a storage service (Google Drive) available for the users via the computer's file system. In order to do so, we will be required to authenticate our account. Notice that we can mount a Google Drive account different from the Google account from which we are running Google Colab.

If you see "Mounted at /content/drive", it means that Google Drive was mounted successfully. After that, you should be able to find a new directory called gdrive in the Colab file explorer.

3. Clone GitHub repository to Colab Runtime system

```
if COLAB:
    import json

    with open("/content/gdrive/My Drive/Git/git.json", "r") as f:
        parsed_json = json.load(f)

    GIT_USER_NAME = parsed_json["GIT_USER_NAME"]
    GIT_TOKEN = parsed_json["GIT_TOKEN"]
    GIT_USER_EMAIL = parsed_json["GIT_USER_EMAIL"]

    GIT_PATH = (
        f"https://{GIT_TOKEN}@github.com/{GIT_USER_NAME}/{GIT_REPOSITORY}.git"
    )

    %cd "{PARENT_DIRECTORY_PATH}"

    !git clone "{GIT_PATH}"  # Clone the github repository

    %cd "{PROJECT_PATH}"
```

medium_python_workflow_for_interactive_projects_2.py hosted with ❤ by GitHub view raw

Here we clone our GitHub repository to the personal Virtual Machine initiated when we started the Colab session. Remember that it has a maximum lifetime of 12 hours.

Notice we will need to pass some credentials in case the GitHub repository to clone is private. In order to keep the credentials safe from the public (we do not want everyone to have access to our private repositories), we will store them in a file (git.json) in our private Google Drive account. In this way, the credentials will only be accessible during the Colab session if you have access to the Google Drive account.

How to generate our credentials file

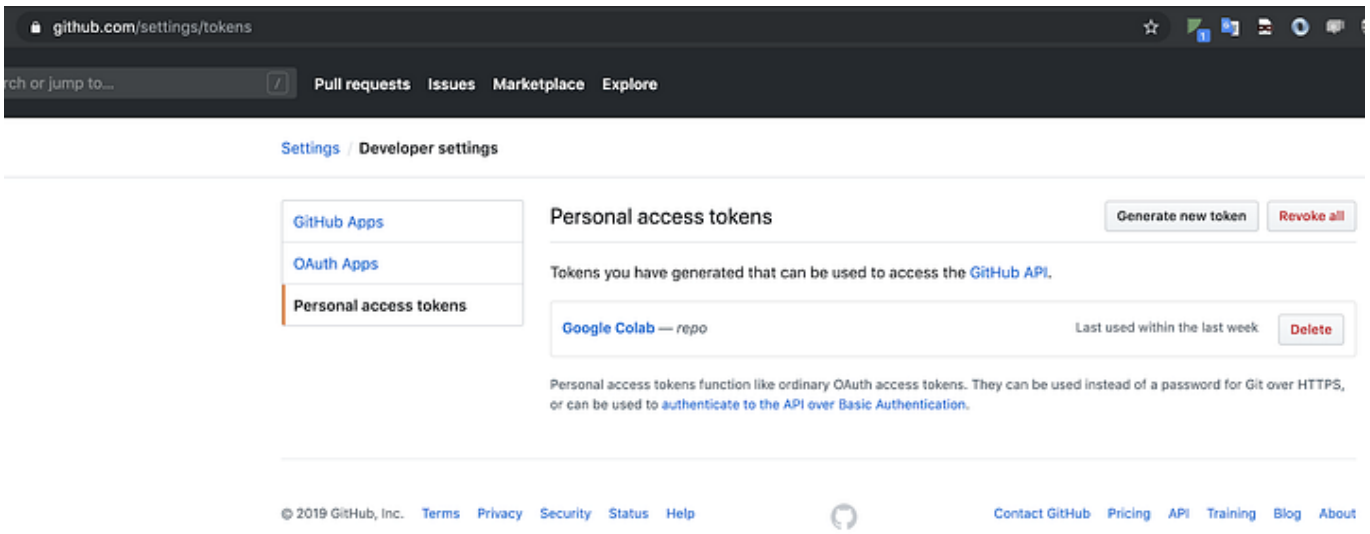
Go to your Google Drive account. Create a directory called Git. Inside this directory create a file called git.json. The file needs to look like this:

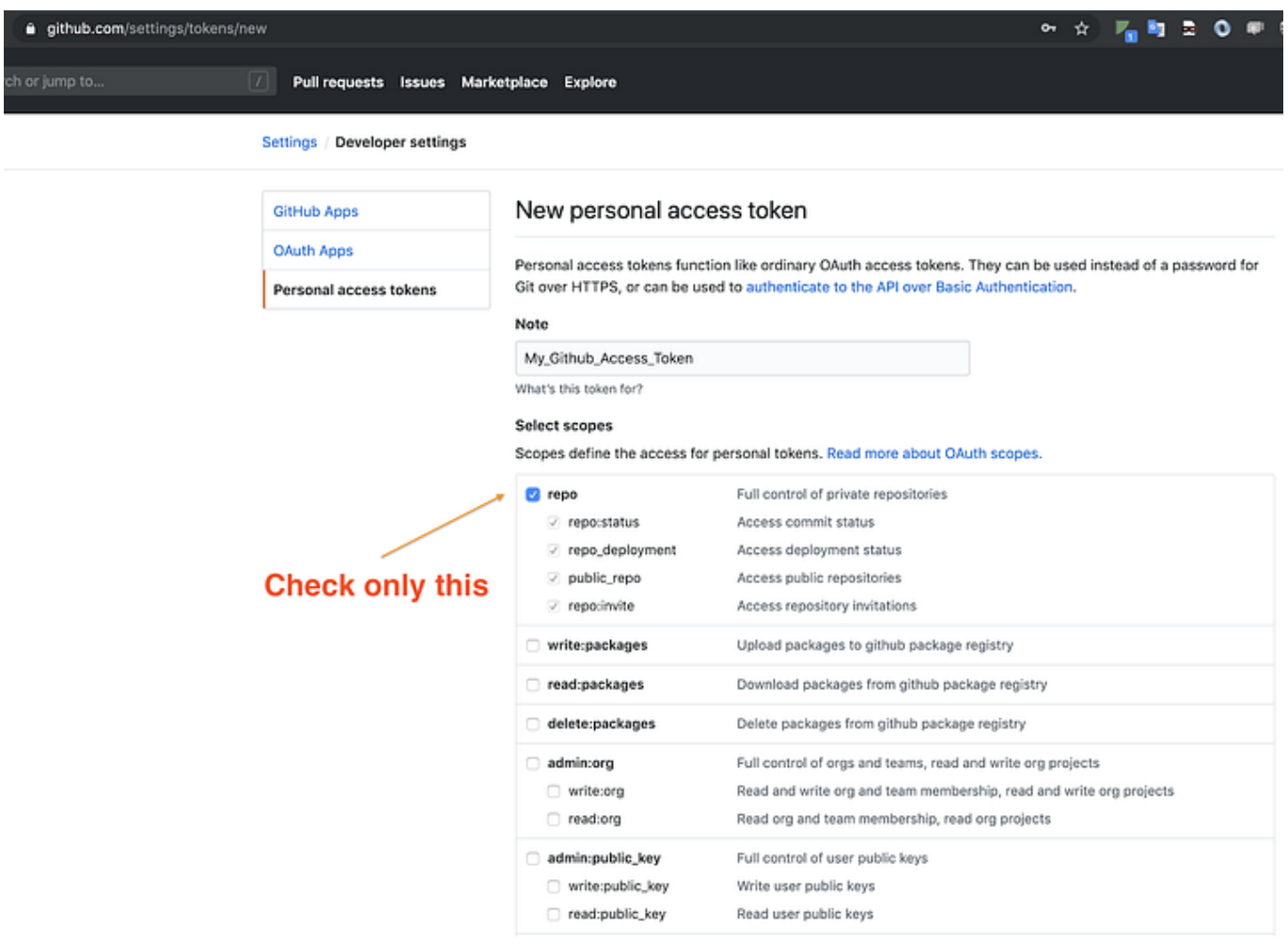
```
{
  "GIT_USER_NAME": "my_git_user_name",
  "GIT_TOKEN": "my_git_token",
  "GIT_USER_EMAIL": "my_git_user_email"
}
```

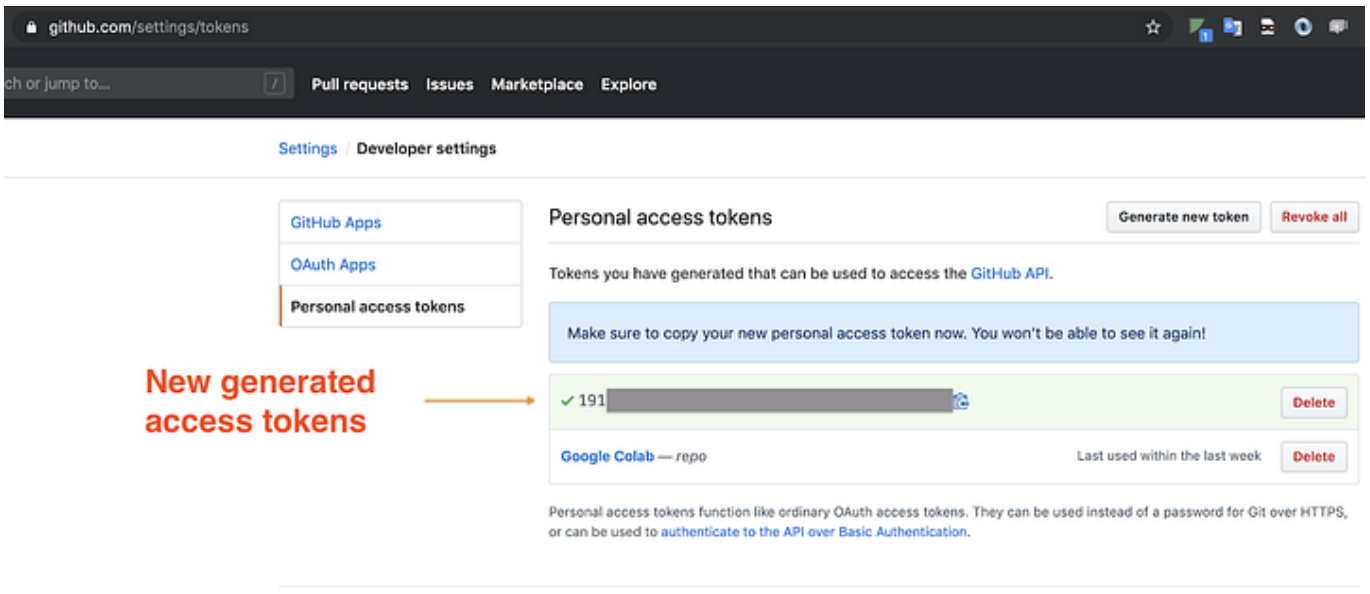
git.json hosted with ❤ by GitHub view raw

In case you do not have a GitHub access token, generate it as follows:

Go to user profile at the top right corner → click on settings → then choose developer settings.







Replace the credentials into the git.json file.

Once you have the credentials ready, you should be able to clone the repository successfully. If so, you will find a new directory with the name of the repository in the Colab file explorer once you execute the cell.

We can also clone our repository on Google Drive. If you prefer to do so, just uncomment the line of code that is mentioned in the feature toggles snippet. Following this second option, you can save your modifications in your Google Drive synchronously. However, I prefer to push my changes to GitHub and clone the repository in each session. This becomes more practical if you are also running your notebooks locally using your IDE.

In case you want to run the Notebook locally, you need to copy the git.json file we generated in your home directory.

4. Kaggle API Setup

(Not necessary if you do not plan to use Kaggle)

```
if COLAB:
    import os
    os.environ["KAGGLE_CONFIG_DIR"] = "/content/gdrive/My Drive/Kaggle"
```

medium_python_workflow_for_interactive_projects_3.py hosted with ❤ by GitHub view raw

Similar to what we did in the previous step, here we pass the credentials of Kaggle in order to use its API.

Go to your Google Drive account. Create a directory called Kaggle. Inside this directory create a file called kaggle.json. The file needs to look like this:

```
{
  "username": "kaggle_user_name",
  "key": "kaggle_token"
}
```

kaggle.json hosted with ❤ by GitHub view raw

In case you do not have a Kaggle API token, generate it as follows [6]:

Sign in to your Kaggle account. From the site header, click on your user profile picture, then on “My Account” from the dropdown menu. This will take you to your account settings at https://www.kaggle.com/account. Scroll down to the section of the page labelled API. To create a new token, click on the “Create New API Token” button. This will download a fresh authentication token onto your machine.

Replace the credentials into the kaggle.json file.

In case you want to run the Notebook locally, you need to copy the kaggle.json file we generated in your home directory.

5. Download competitions data using the Kaggle API

(Not necessary if you do not plan to use Kaggle)

```
if DOWNLOAD_DATA:
    !mkdir input
    %cd input

    !pip install --upgrade kaggle
    # Go to kaggle and copy the API Command to download the dataset
    # !kaggle competitions download -c {GIT_REPOSITORY}
    # Instead of downloading all data, we select specific files.
    !kaggle competitions download {GIT_REPOSITORY} -f train.csv
    !kaggle competitions download {GIT_REPOSITORY} -f test.csv
    !kaggle competitions download {GIT_REPOSITORY} -f sample_submission.csv

    # Unzipping the zip files and deleting the zip files
    !unzip \*.zip  && rm *.zip

    # After downloading all data, go back to PROJECT_PATH
    %cd ..

if KAGGLE:
    INPUT_ROOT = f"../input/{GIT_REPOSITORY}"
else:
    INPUT_ROOT = f"../{GIT_REPOSITORY}/input"

# INPUT_ROOT will be used as follows
# train_df = pd.read_csv(f"{INPUT_ROOT}/train.csv")
```

medium_python_workflow_for_interactive_projects_4.py hosted with ❤ by GitHub view raw

Here we create a new directory called input inside the VM and we download the data of the requested competition. Notice the data is stored in the Colab Runtime system so it does not occupy space in your Drive.

As you can observe, here I am reusing the GIT_REPOSITORY name to download the data. This is because my repository is called as the competition: “osic-pulmonary-fibrosis-progression”. If this is not your case, just replace the lines with the original Kaggle API commands from the competition webpage.

If successful, we should see a new directory called input inside the cloned repository, and all the data stored in it.

Keep in mind that you do not want to push your input folder into GitHub, so you should have a .gitignore file with input/ in it.

6. Save changes to GitHub

Remember that we can run all git commands from here. Therefore, we can automate the process of saving to GitHub.

```
if SAVE_TO_GITHUB:
    !git add {FILE_NAME}
    !git config --global user.email {GIT_USER_EMAIL}
    !git config --global user.name {GIT_USER_NAME}
    !git commit -am "update {FILE_NAME}"
    !git push
```

medium_python_workflow_for_interactive_projects_5.py hosted with ❤ by GitHub view raw

Notice that FILE_NAME should coincide with the name of the notebook. Even though this is useful, I usually do not use it unless I need to let my whole pipeline running and want to save the changes after it. The alternative is to save it directly using the tools from Colab: Go to File, then Save a copy in GitHub. This will commit and push your changes to GitHub.

The workflow

Now that we have everything set up, let me describe briefly how my workflow usually looks like:

- Create a repository.
- Start a new notebook from Colab.
- Copy paste all the mentioned cells.
- Configure the feature toggles and constants.
- Work on the notebook.
- Save changes to GitHub continuously.

After that,

- Once I start a session again, upload the notebook from GitHub using the Colab toolbar.
- If I need to debug, I clone the repo locally, pull and open the notebook with my preferred IDE (PyCharm).

References

[1] “Python Developers Survey 2019 Results”. JetBrains. https://www.jetbrains.com/lp/python-developers-survey-2019/.

[2] Project Jupyter. https://jupyter.org/about.

[3] Google. https://research.google.com/colaboratory/faq.html#gpu-availability.

[4] Google. https://cloud.google.com/tpu/docs/tpus.

[5] Kaggle. https://www.kaggle.com/docs/notebooks.

[6] Kaggle. https://www.kaggle.com/docs/api.

Jupyter Notebook | Google Colab | Python | Kaggle | Data Science

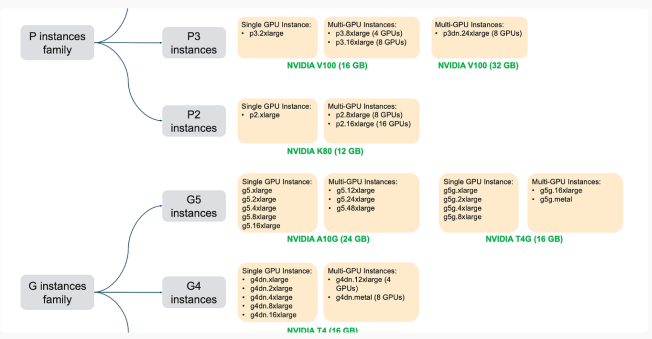
Written by Aleix López Pascual

Follow

46 Followers · Writer for Analytics Vidhya

Senior Data Scientist @ Glovo | Competitions Expert @ Kaggle | Writer @ Medium | MSc in High Energy Physics, Astrophysics and Cosmology

More from Aleix López Pascual and Analytics Vidhya



Aleix López Pascual

Cloud GPU instances to solve out of memory error 2022

Discover the GPUs with the largest VRAM

3 min read · Oct 14, 2022

59

Sajal Rastogi in Analytics Vidhya

Wifi -Hacking using PyWifi

.

4 min read · Feb 6, 2021

257 1

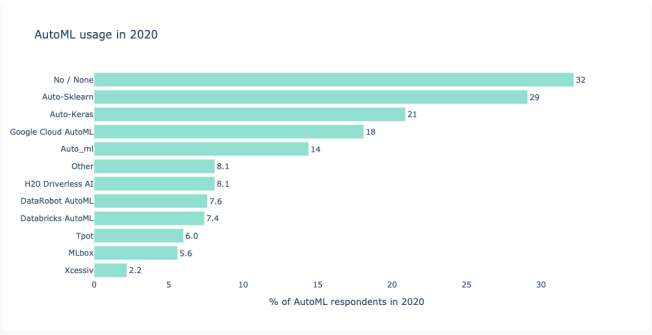
Kia Eisinga in Analytics Vidhya

How to create a Python library

Ever wanted to create a Python library, albeit for your team at work or for some open...

7 min read · Jan 27, 2020

2K 24



Aleix López Pascual

Comparison of AutoML solutions 2021

Learn about the current state of AutoML solutions, adoption, market size and which...

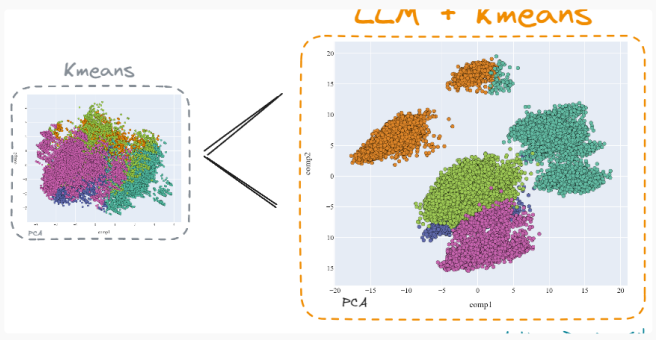
7 min read · Aug 19, 2021

197 1

See all from Aleix López Pascual

See all from Analytics Vidhya

Recommended from Medium



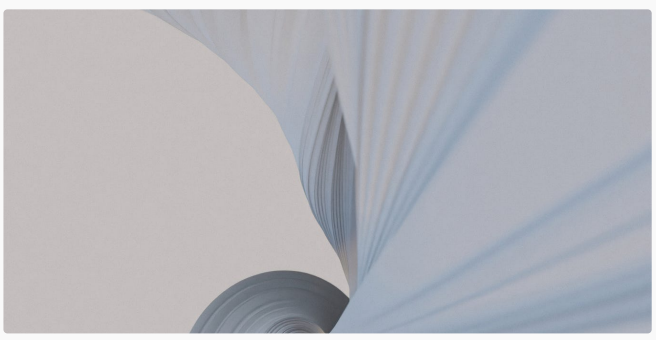
Damian Gil in Towards Data Science

Mastering Customer Segmentation with LLM

Unlock advanced customer segmentation techniques using LLMs, and improve your...

23 min read · Sep 27

2.6K 24



Abel K Widodo

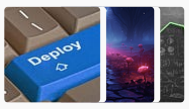
Exploring the Art of Generative AI in Python

Artificial Intelligence (AI) has transformed various industries, and one fascinating area ...

6 min read · Jul 15

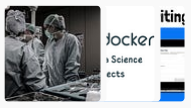
26

Lists



Predictive Modeling w/ Python

20 stories · 459 saves



Coding & Development

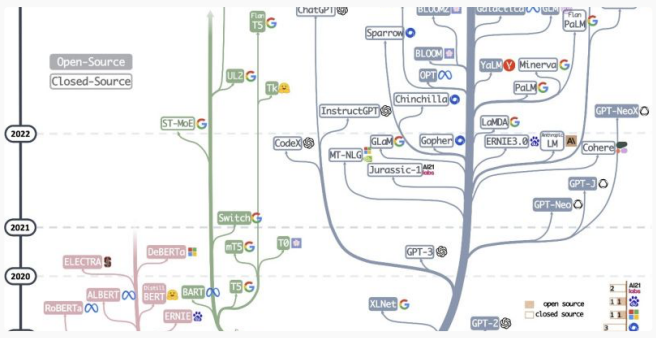
11 stories · 202 saves

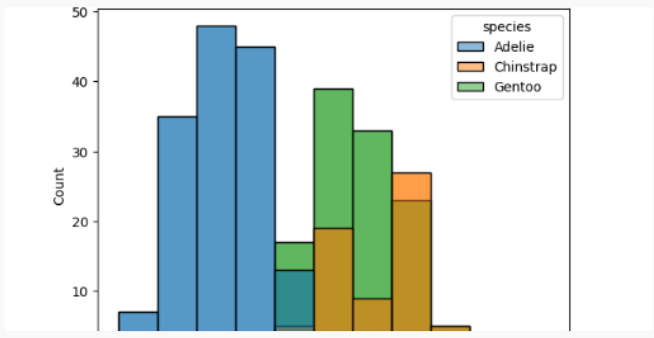
New_Reading_List

174 stories · 136 saves

Practical Guides to Machine Learning

10 stories · 530 saves





Haifeng Li

A Tutorial on LLM

Generative artificial intelligence (GenAI), especially ChatGPT, captures everyone's...

15 min read · Sep 14

544

Kok Hua

PandasAI—Exploratory Data Analysis with Pandas and AI...

Discovering PandasAI: Exploring Pandas with Generative AI Capabilities

3 min read · Jun 16

9

Hugging F

Aitor Porcel Laburu

Run open-source LLM for free in Google Colab/Kaggle

Over the past year, LLMs have become increasingly popular for their ability to...

2 min read · Jun 28

13

Bright Eshun

Building A Machine Learning API with FastAPI

1. Introduction

12 min read · Jun 10

See more recommendations